

Employee Job Review Analysis

Data Mining Project Report

Prepared by: Mohamed Aziz Amri, Mohamed Ben Arbia, Rissen Guermazi

Course title: Data Mining

Video presentation link: <https://canva.link/v3fozz3l55wvso9>

Semester: Spring 2026

1.Dataset Description

The dataset contains +838586 employee reviews describing experiences at companies. It is a text mining dataset in which employees write about what they liked and disliked at work. In the notebook, the CSV file is loaded as `glassdoor_reviews.csv`, and the presentation identifies it as a Glassdoor job reviews dataset hosted on Kaggle.

Each observation represents one employee review. After loading and cleaning, the notebook keeps the following raw variables: `company`, `pros`, `cons`, and `rating`. The company name identifies the employer; `pros` contains positive comments; `cons` contains negative comments; and `rating` is the overall score on a 1-to-5 scale.

The dataset is useful for sentiment analysis because it combines free-text opinions with a structured numerical rating. This makes it suitable for both supervised learning, where the rating is used to define sentiment labels, and unsupervised learning, where the text is explored for recurring themes.

Main variables used in the project:

- `company`: company name associated with the review.
- `pros`: positive aspects written by the employee.
- `cons`: negative aspects written by the employee.
- `rating`: overall review score from 1 to 5.
- `combined`: engineered text field created by merging `pros` and `cons`.
- `sentiment_score`: VADER compound sentiment value.
- `topic`: dominant LDA topic assigned to the review.

A small preview of the cleaned text is shown below to illustrate the effect of preprocessing.

```
friendly welcoming new staff easy going ethic ...  
friendly helpful hard working colleague poor s...  
easy even without experience finance low salar...  
nice staff career progression salary poor  
easy nice colleague abysmal pay around minimum...
```

2. Problem Statement

The problem studied in this project is how to automatically analyze employee reviews that are written in unstructured text. Manually reading large numbers of reviews is slow and difficult, so the goal is to extract useful patterns from the text and turn them into actionable insights.

The main objective of the analysis is to combine text preprocessing, sentiment analysis, topic modeling, and machine learning to understand what employees are saying and to classify reviews by sentiment. The notebook also explores a hybrid decision rule that combines a lexicon-based sentiment score with a machine learning prediction.

The main research questions are:

- What sentiment tendency appears in the employee reviews?
- Which themes are discussed most often in the reviews?
- Can a logistic regression model classify reviews as positive or negative from text alone?
- Does combining VADER with the classifier produce a more balanced final review judgment?

The target variable for the classification task is the sentiment label derived from the numeric rating. In the notebook, reviews with rating 4 or 5 are labeled positive, while reviews with rating 1 or 2 are treated as negative. Reviews with rating 3 are excluded from the classifier to keep the task binary and clearer for evaluation.

3. Data Preparation

The data preparation stage starts by reading the CSV file, standardizing the column names, keeping the relevant fields, removing missing rows, converting the rating to numeric form, and filtering the valid rating range from 1 to 5. This is necessary because the analysis depends on both clean text and a reliable rating column.

The notebook does not perform explicit outlier treatment because the main variables are textual and ordinal rather than continuous numeric measurements. Instead, the main cleaning step is the removal of incomplete or invalid records, which is the correct choice for this dataset structure.

For text preprocessing, the notebook combines the pros and cons fields into one text variable and applies several transformations: lowercasing, removal of non-letter characters, tokenization, stopwords filtering, lemmatization, and the addition of domain-specific stopwords such as company, work, job, people, get, make, and also. This improves the quality of the vocabulary by removing very common or low-value terms.

The preprocessing choices are justified because text mining models are sensitive to noisy words, punctuation, and word variants. Lemmatization helps reduce inflected forms to a common root, and removing stopwords reduces dimensionality without losing important meaning.

```
df = pd.read_csv("glassdoor_reviews.csv")

df.columns = df.columns.str.lower().str.replace("-", "_")

df = df[["firm", "pros", "cons", "overall_rating"]].dropna()
df.columns = ["company", "pros", "cons", "rating"]

df["rating"] = pd.to_numeric(df["rating"], errors="coerce")
df = df[df["rating"].between(1, 5)]

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words("english"))

extra = {"company", "work", "job", "people", "get", "make", "also", "really"}
stop_words.update(extra)

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = text.split()
    tokens = [
        lemmatizer.lemmatize(t)
        for t in tokens
        if t not in stop_words and len(t) > 2
    ]
    return " ".join(tokens)

df["combined"] = (df["pros"] + " " + df["cons"]).apply(clean_text)
```

Figure 1 shows the distribution of the ratings after cleaning.

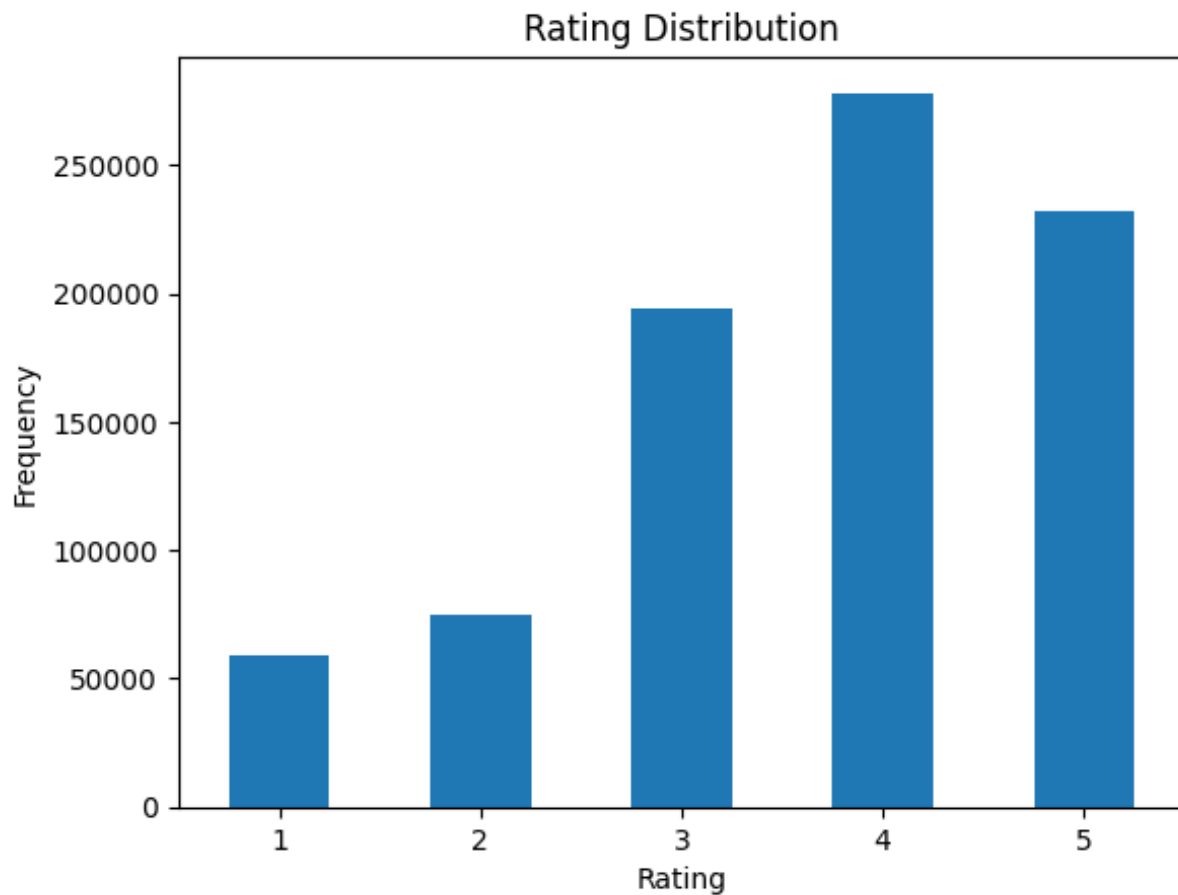


Figure 1. Rating distribution

The rating distribution is clearly skewed toward higher scores, especially 4 and 5. This indicates a positive bias in the dataset and explains why the classification task is imbalanced. The imbalance is important because it affects model evaluation: accuracy alone may look strong even when the minority class is harder to identify.

4. Methods and Analysis

The project uses four main methods: VADER sentiment analysis, TF-IDF feature extraction, LDA topic modeling, and logistic regression classification. These methods are appropriate because the data are text-heavy and the project needs both exploratory understanding and predictive modeling.

4.1 Sentiment analysis with VADER

VADER is a lexicon-based sentiment tool that produces a compound score between -1 and +1. It is useful as a fast baseline because it captures the emotional tone of short informal text without training a separate model.

```
analyzer = SentimentIntensityAnalyzer()

df["sentiment_score"] = df["combined"].apply(
    lambda x: analyzer.polarity_scores(x)["compound"]
)
```

Figure 2 shows the sentiment score distribution.

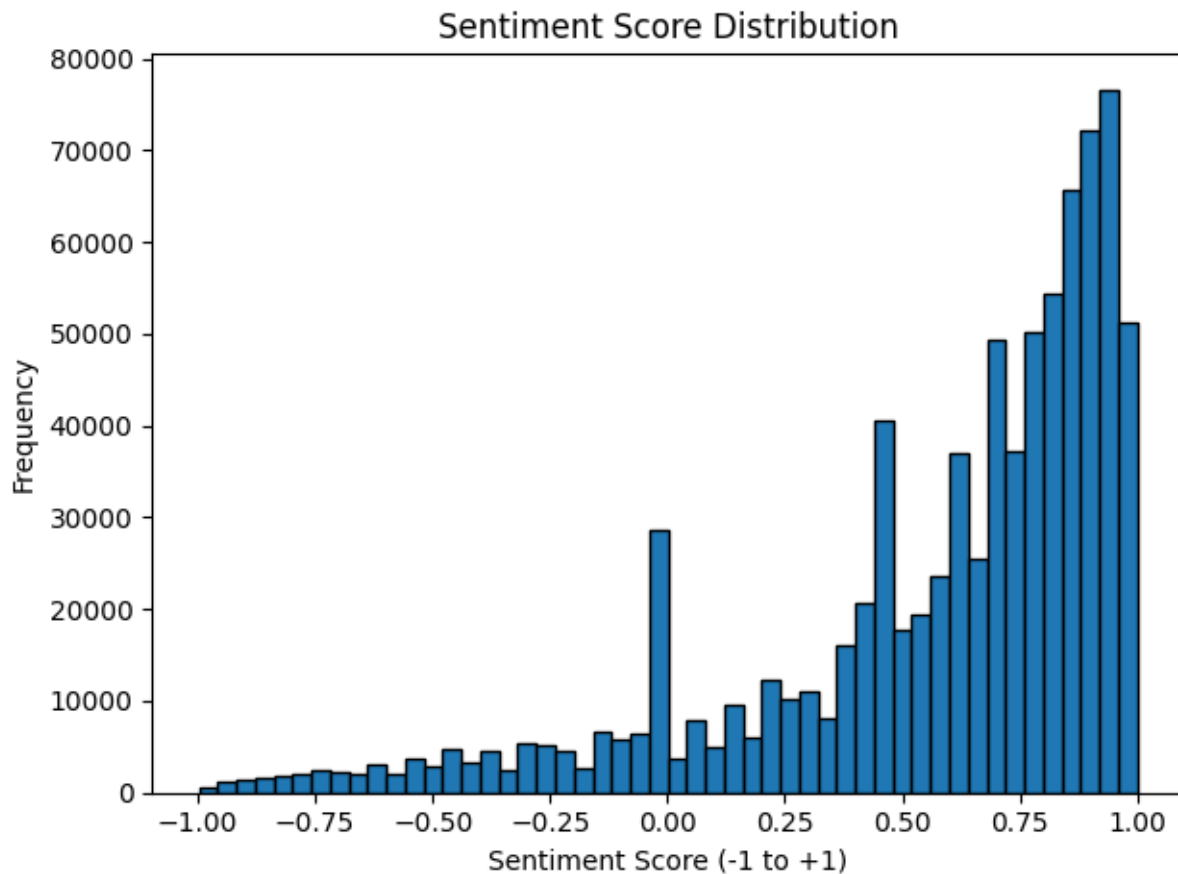


Figure 2. Sentiment score distribution

The histogram is concentrated on the positive side of the scale, showing that most reviews in the dataset are expressed positively. At the same time, the existence of scores near -1 confirms that negative opinions are also present. This supports the idea that employee feedback is mixed, even if the overall sentiment is positive.

4.2 TF-IDF feature engineering

The text is transformed into numerical features using TF-IDF. This method is appropriate for text classification because it gives higher weight to informative words and lower weight to very common ones. The notebook uses up to 3000 TF-IDF features and includes unigrams and bigrams to capture short phrases.

```
tfidf = TfidfVectorizer(max_features=3000, ngram_range=(1,2), min_df=5)
X_tfidf = tfidf.fit_transform(df["combined"])
```

4.3 Topic modeling with LDA

Latent Dirichlet Allocation is used to discover hidden themes in the employee reviews. This unsupervised method is appropriate because it helps summarize the main concerns mentioned in the text without requiring labels.

```
print("\n--- Preparing LDA ---")

vectorizer_lda = CountVectorizer(max_features=5000, stop_words="english")
X_lda = vectorizer_lda.fit_transform(df["combined"])

print(f"Vocabulary size: {len(vectorizer_lda.get_feature_names_out())}")
```

```

lda_model = LatentDirichletAllocation(
    n_components=5,
    random_state=42,
    learning_method="batch",
    max_iter=5
)

lda_model.fit(X_lda)

def print_topics(model, feature_names, n_words=8):
    print("\n--- LDA Topics ---")
    for idx, topic in enumerate(model.components_):
        top_words = [feature_names[i] for i in topic.argsort()[-n_words:][::-1]]
        print(f"\nTopic {idx + 1}: " + " ".join(top_words))

print_topics(lda_model, vectorizer_lda.get_feature_names_out())

df["topic"] = lda_model.transform(X_lda).argmax(axis=1)

```

Topic	Top keywords
Topic 1	great, place, customer, environment, employee, team, lot, working
Topic 2	management, good, hour, staff, pay, manager, time, working
Topic 3	good, hour, life, working, balance, environment, great, salary
Topic 4	opportunity, great, good, management, life, team, culture, balance
Topic 5	great, year, benefit, time, good, lot, employee, like

Table 1. Main LDA topics extracted from the review corpus.

Figure 3 shows how frequently each topic appears in the dataset.

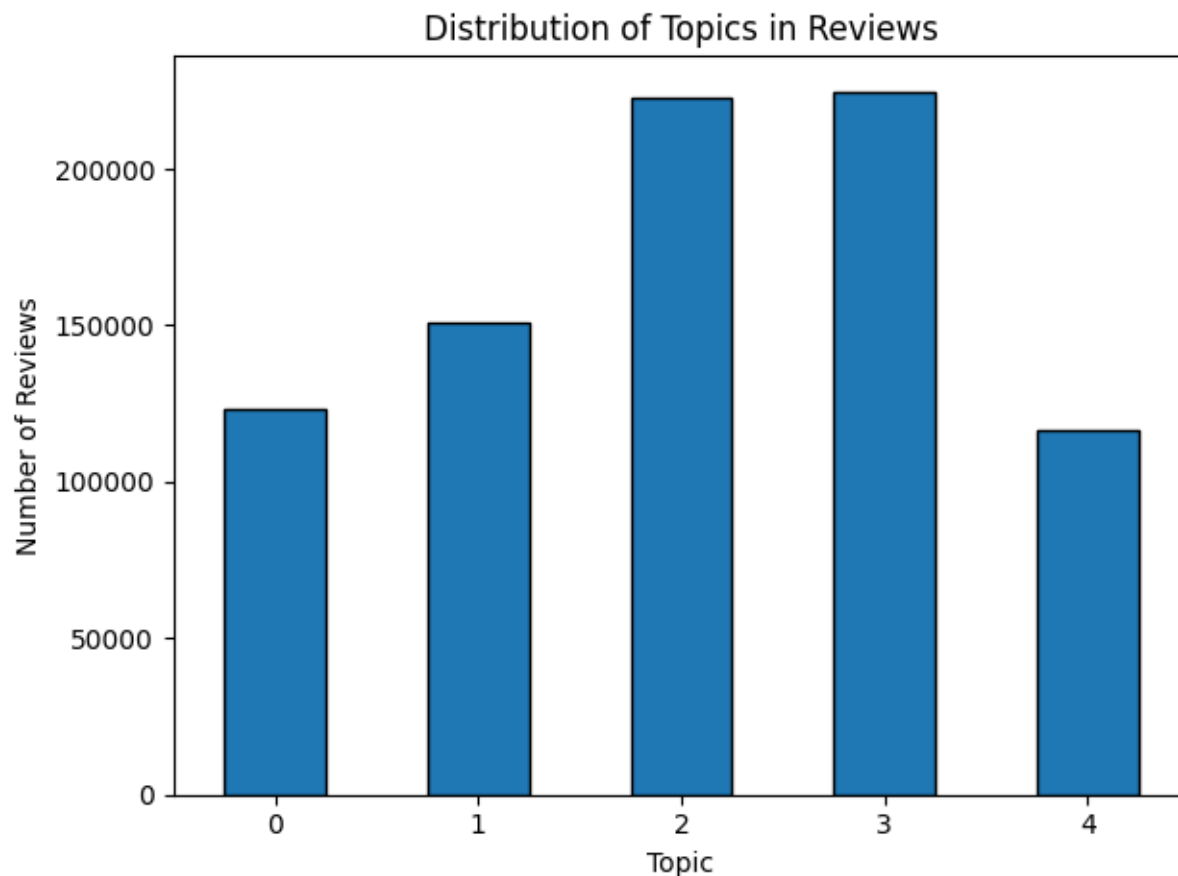


Figure 3. Topic distribution

Topics 2 and 3 are the most frequent. Their keywords point to management, working hours, pay, salary, and work-life balance. This indicates that employees are especially focused on practical workplace conditions rather than only general satisfaction. In other words, compensation and management quality appear to be the strongest recurring concerns in the review text.

4.4 Logistic regression classification

For the predictive task, the notebook builds a binary classifier. Reviews with rating 3 are removed, then the remaining reviews are labeled positive when the rating is at least 4 and negative otherwise. Logistic regression is a strong choice here because it works well with sparse high-dimensional TF-IDF vectors and remains easy to interpret.

```
df_clf = df[df["rating"] != 3].copy()
df_clf["label"] = (df_clf["rating"] >= 4).astype(int)

X = tfidf.transform(df_clf["combined"])
y = df_clf["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

lr = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    solver="liblinear"
)
```

```
lr.fit(X_train, y_train)

pred_lr = lr.predict(X_test)

print("LOGISTIC REGRESSION RESULTS")
print(classification_report(y_test, pred_lr))
```

Class	Precision	Recall	F1-score	Support
Negative (0)	0.56	0.80	0.66	26,753
Positive (1)	0.94	0.84	0.89	102,104
Accuracy			0.83	128,857
Macro avg	0.75	0.82	0.77	128,857
Weighted avg	0.86	0.83	0.84	128,857

Table 2. Logistic regression performance on the test set.

The confusion matrix below shows where the classifier makes mistakes.

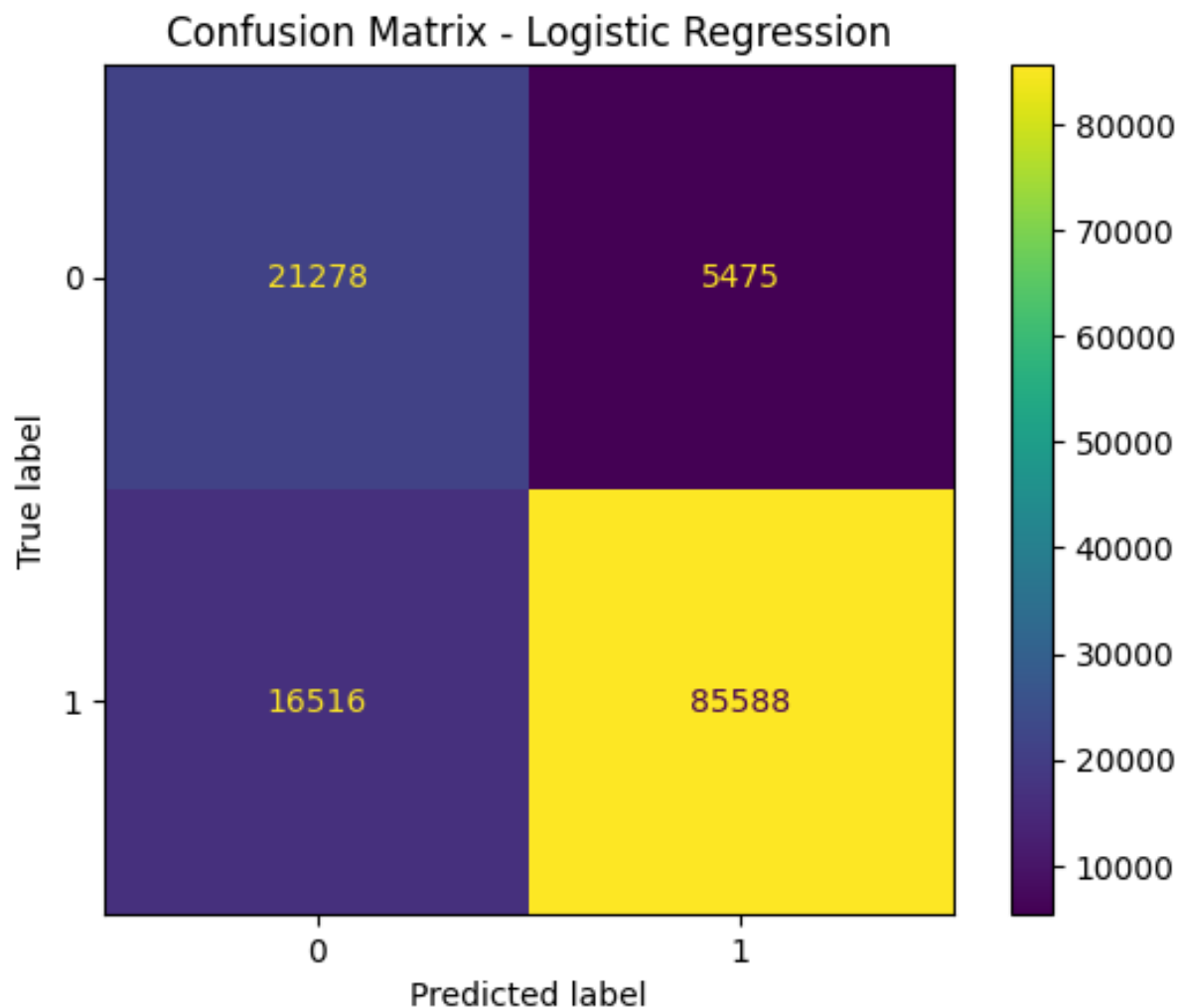


Figure 4. Confusion matrix for logistic regression

The model performs better on positive reviews than on negative ones, which is expected given the imbalance in the data. The lower precision for the negative class means that some positive reviews are incorrectly predicted as negative, while the lower recall for negatives means that some truly negative reviews are

missed. This is why the `class_weight='balanced'` option is useful: it helps the model pay more attention to the minority class.

```
from sklearn.model_selection import cross_val_score

lr_cv_scores = cross_val_score(
    lr,
    tfidf.transform(df_clf["combined"]),
    df_clf["label"],
    cv=3,
    scoring="f1",
    n_jobs=-1
)

print("LOGISTIC REGRESSION CROSS-VALIDATION")
print(f"F1 Score: {lr_cv_scores.mean():.3f} ± {lr_cv_scores.std():.3f}")
```

The cross-validation result of $F1 = 0.884 \pm 0.010$ suggests that the model is stable across folds. The small standard deviation means that the performance does not depend heavily on a single train-test split, which strengthens confidence in the classifier.

4.5 Hybrid review judge

The final part of the notebook combines the machine learning classifier with VADER in a judge-style function. This step is valuable because VADER can capture immediate sentiment cues, while logistic regression learns from the dataset patterns. The fusion provides a more interpretable final decision for a single review.

```
def predict_review(review_text):

    text = clean_text(review_text)

    # VADER
    vader_score = analyzer.polarity_scores(review_text)["compound"]

    if vader_score >= 0.05:
        vader_label = "Positive ☐"
    elif vader_score <= -0.05:
        vader_label = "Negative ☐"
    else:
        vader_label = "Mixed / Neutral ☐"

    # ML prediction
    vec = tfidf.transform([text])
    ml_pred = lr.predict(vec)[0]
    ml_proba = lr.predict_proba(vec)[0]

    ml_label = "Positive ****" if ml_pred == 1 else "Negative ☐"

    # Fusion
    ml_score = ml_proba[1] - ml_proba[0]
    final_score = (0.6 * vader_score) + (0.4 * ml_score)

    if final_score > 0.1:
        final_label = "FINAL: Positive ✔"
    elif final_score < -0.1:
        final_label = "FINAL: Negative ✖"
    else:
        final_label = "FINAL: Mixed ☐"

    # Output
    print("\n" + "=" * 60)
    print("☐ SENTIMENT JUDGE SYSTEM")
    print("=" * 60)

    print(f"Input Review: {review_text}")
```

```

print("\n--- VADER ---")
print(f"Score: {vader_score:+.3f} → {vader_label}")

print("\n--- ML Model ---")
print(f"Prediction: {ml_label}")

print("\n--- FINAL DECISION ---")
print(final_label)

print("=" * 60)

```

The example test review 'poor management but flexible hours and good colleagues' is classified as positive by VADER, negative by the machine learning model, and mixed by the fusion rule. This is a good demonstration of why the hybrid approach is useful: it can better handle reviews that contain both positive and negative statements.

```
predict_review("poor management but flexible hours and good colleagues")
```

5. Results and Interpretation

The main results can be summarized as follows. First, the rating distribution shows a clear positive bias, meaning that many reviews are rated highly. Second, VADER sentiment scores are mostly positive, confirming that the textual tone of the reviews is generally favorable. Third, LDA reveals that management, salary, working hours, and work-life balance are repeated themes. Finally, logistic regression achieves strong overall performance, although it still struggles more with the negative class.

Key result	Interpretation
Positive class dominates	The dataset is imbalanced and contains more favorable reviews than unfavorable ones.
Sentiment distribution is right-skewed	Most combined review texts are positive after VADER scoring.
Topic 2 and Topic 3 dominate	Employees focus strongly on management, hours, salary, and balance.
Accuracy = 0.83	The logistic regression model is useful, but accuracy should be read together with class-wise metrics.
F1 cross-validation = 0.884 ± 0.010	The classifier is stable across multiple folds.

Table 3. Interpretation of the main results.

Overall, the analysis suggests that the company reviews are mostly positive, but the recurring negative concerns are not random. They are concentrated around management quality, pay, and working conditions. These are the practical issues that a company can act on most directly.

6. Conclusion

This project aimed to analyze employee reviews and extract useful insights from unstructured text. The notebook followed a complete text mining workflow: dataset cleaning, text preprocessing, sentiment scoring, topic modeling, and binary classification with logistic regression.

The most important findings are that the dataset is positively biased, the text contains strong signals about management and compensation, and the logistic regression model performs well enough to classify reviews with solid stability. The hybrid VADER-plus-ML function adds an interpretable layer that is useful for single-review analysis.

In practical terms, this type of analysis can help a company monitor employee satisfaction, detect recurring complaints, and identify HR issues earlier. The project shows that text mining can transform raw employee comments into structured and actionable knowledge.

Notebook Code Blocks in Order

This appendix groups the main code blocks from the notebook in the same general order as the original analysis. The blocks are included so the report remains editable and easy to compare with the notebook.

1. Libraries and setup

```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re

import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import classification_report
```

2. NLTK downloads

```
nltk.download("stopwords")
nltk.download("wordnet")
```

3. Data loading and cleaning

```
df = pd.read_csv("glassdoor_reviews.csv")

df.columns = df.columns.str.lower().str.replace("-", "_")

df = df[["firm", "pros", "cons", "overall_rating"]].dropna()
df.columns = ["company", "pros", "cons", "rating"]

df["rating"] = pd.to_numeric(df["rating"], errors="coerce")
df = df[df["rating"].between(1, 5)]
```

4. Rating distribution

```
rating_counts = df["rating"].value_counts().sort_index()

rating_counts.plot(kind="bar")

plt.title("Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Frequency")

plt.xticks(rotation=0)

plt.show()
```

5. Text preprocessing

```
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words("english"))

extra = {"company", "work", "job", "people", "get", "make", "also", "really"}
stop_words.update(extra)

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"^[a-z\s]", " ", text)
    tokens = text.split()
```

```

        tokens = [
            lemmatizer.lemmatize(t)
            for t in tokens
            if t not in stop_words and len(t) > 2
        ]
        return " ".join(tokens)

df["combined"] = (df["pros"] + " " + df["cons"]).apply(clean_text)

```

6. Sentiment scoring

```

analyzer = SentimentIntensityAnalyzer()

df["sentiment_score"] = df["combined"].apply(
    lambda x: analyzer.polarity_scores(x)["compound"]
)

```

7. Sentiment histogram

```

df["sentiment_score"].plot(
    kind="hist",
    bins=50,
    edgecolor="black"
)

plt.title("Sentiment Score Distribution")
plt.xlabel("Sentiment Score (-1 to +1)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

```

8. TF-IDF

```

tfidf = TfidfVectorizer(max_features=3000, ngram_range=(1,2), min_df=5)
X_tfidf = tfidf.fit_transform(df["combined"])

```

9. LDA topic modeling

```

print("\n--- Preparing LDA ---")

vectorizer_lda = CountVectorizer(max_features=5000, stop_words="english")
X_lda = vectorizer_lda.fit_transform(df["combined"])

print(f"Vocabulary size: {len(vectorizer_lda.get_feature_names_out())}")

lda_model = LatentDirichletAllocation(
    n_components=5,
    random_state=42,
    learning_method="batch",
    max_iter=5
)

lda_model.fit(X_lda)

def print_topics(model, feature_names, n_words=8):
    print("\n--- LDA Topics ---")
    for idx, topic in enumerate(model.components_):
        top_words = [feature_names[i] for i in topic.argsort()[-n_words:][::-1]]
        print(f"\nTopic {idx + 1}: " + " ".join(top_words))

print_topics(lda_model, vectorizer_lda.get_feature_names_out())

df["topic"] = lda_model.transform(X_lda).argmax(axis=1)

```

10. Topic distribution

```

df["topic"].value_counts().sort_index().plot(
    kind="bar",
    edgecolor="black"
)

plt.title("Distribution of Topics in Reviews")
plt.xlabel("Topic")

```

```
plt.ylabel("Number of Reviews")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

11. Classification split

```
df_clf = df[df["rating"] != 3].copy()
df_clf["label"] = (df_clf["rating"] >= 4).astype(int)

X = tfidf.transform(df_clf["combined"])
y = df_clf["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

12. Logistic regression and report

```
lr = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    solver="liblinear"
)

lr.fit(X_train, y_train)

pred_lr = lr.predict(X_test)

print("LOGISTIC REGRESSION RESULTS")
print(classification_report(y_test, pred_lr))
```

13. Confusion matrix

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, pred_lr)

plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

14. Cross-validation

```
from sklearn.model_selection import cross_val_score

lr_cv_scores = cross_val_score(
    lr,
    tfidf.transform(df_clf["combined"]),
    df_clf["label"],
    cv=3,
    scoring="f1",
    n_jobs=-1
)

print("LOGISTIC REGRESSION CROSS-VALIDATION")
print(f"F1 Score: {lr_cv_scores.mean():.3f} ± {lr_cv_scores.std():.3f}")
```

15. Hybrid predictor

```
def predict_review(review_text):

    text = clean_text(review_text)

    # VADER
    vader_score = analyzer.polarity_scores(review_text)["compound"]

    if vader_score >= 0.05:
        vader_label = "Positive ☐"
    elif vader_score <= -0.05:
        vader_label = "Negative ☐"
    else:
        vader_label = "Mixed / Neutral ☐"
```

```

# ML prediction
vec = tfidf.transform([text])
ml_pred = lr.predict(vec)[0]
ml_proba = lr.predict_proba(vec)[0]

ml_label = "Positive ****" if ml_pred == 1 else "Negative □"

# Fusion
ml_score = ml_proba[1] - ml_proba[0]
final_score = (0.6 * vader_score) + (0.4 * ml_score)

if final_score > 0.1:
    final_label = "FINAL: Positive ✔"
elif final_score < -0.1:
    final_label = "FINAL: Negative ✖"
else:
    final_label = "FINAL: Mixed □"

# Output
print("\n" + "=" * 60)
print("□ SENTIMENT JUDGE SYSTEM")
print("=" * 60)

print(f"Input Review: {review_text}")

print("\n--- VADER ---")
print(f"Score: {vader_score:+.3f} → {vader_label}")

print("\n--- ML Model ---")
print(f"Prediction: {ml_label}")

print("\n--- FINAL DECISION ---")
print(final_label)

print("\n" + "=" * 60)

```

16. Demo call

```
predict_review("poor management but flexible hours and good colleagues")
```